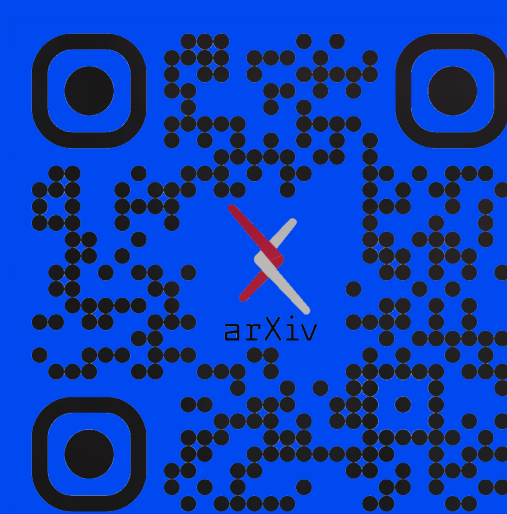




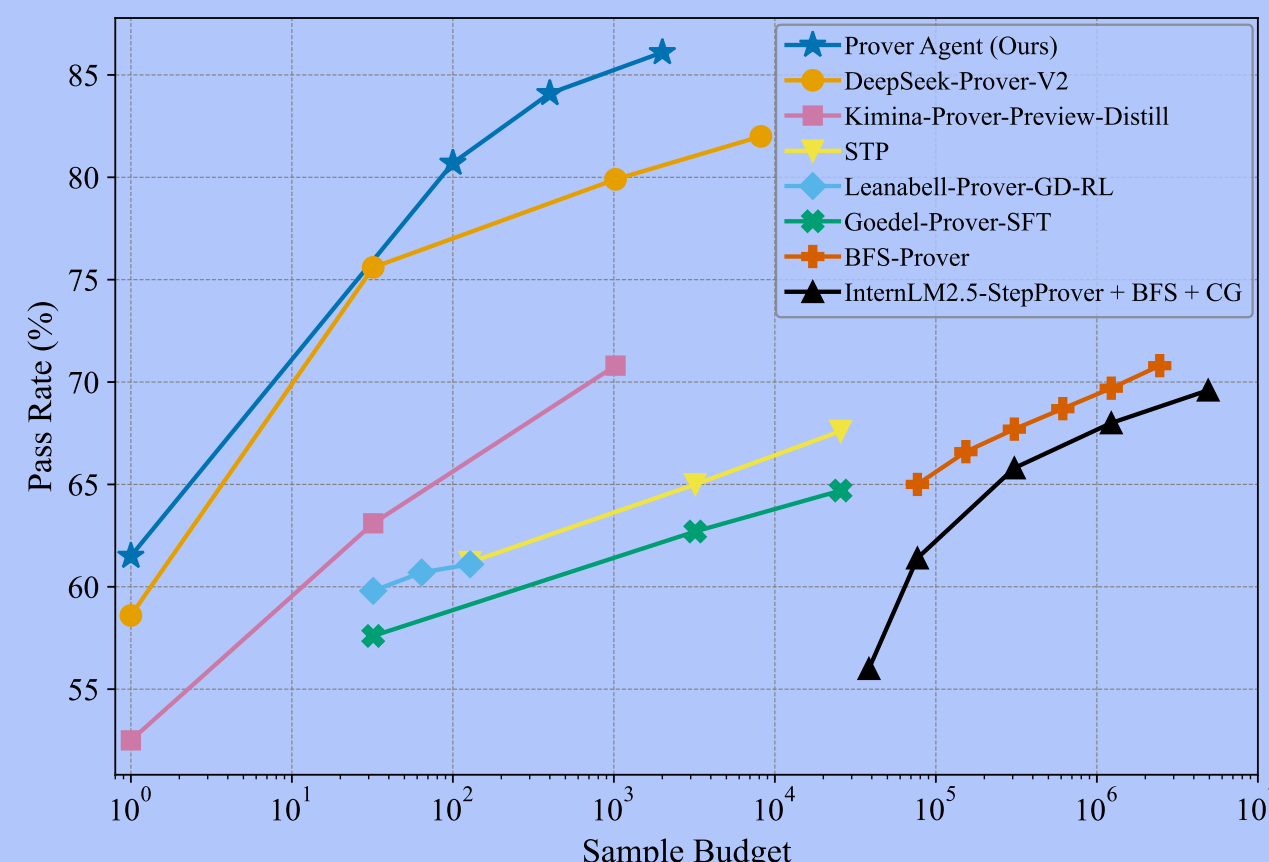
Motivation

- Large language models (LLMs):
 - ✓ Capable of powerful reasoning and generation
 - ✗ Prone to errors and hallucinations
- Formal proof assistants (e.g., Lean):
 - ✓ Verify mathematical correctness
 - ✗ Not generative; requires painstaking meticulous detail
- LLM-based formal proving is gaining attention
- ✗ Yet, a large gap remains between informal reasoning and formal proving

Our Goal: Bridge this gap

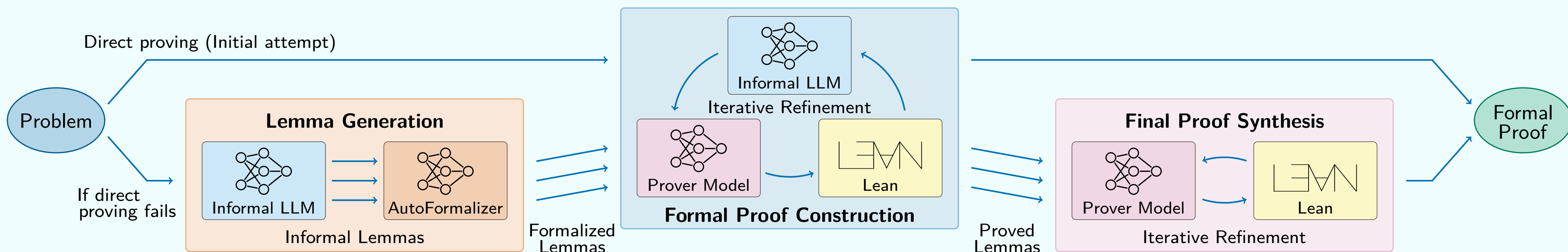
Our Contributions

- Coordination of informal and formal reasoning with Lean feedback
- Auxiliary lemma generation for strategy discovery
 - Helps discover strategies even when the solution path is not apparent at first
- State-of-the-art theorem-proving performance among methods using small language models
- Efficiency in inference-time cost
 - Much smaller sample budget than prior work



Comparison of theorem-proving performance on the MiniF2F benchmark

Prover Agent



Three Key Components of Prover Agent

1 Lemma Generation via Informal Reasoning

- Generate auxiliary lemmas
 - Specific cases
 - Potentially useful intermediate facts
- Not limited to subgoals of predefined proof sketch
 - Key difference from prior approaches
- e.g. Problem: Show that $n^2 + an$ is even
 - Consider $n^2 + n$ or $n^2 + 3n$ ($n \in \mathbb{N}$, a : even)
- Help discover overall proof strategy
- ✓ Mirrors how human mathematicians typically work

2 Formal Proof Construction Guided by Informal Reasoning and Iterative Feedback

- Leverage the stronger mathematical ability of the informal LLM
- Construct a formal proof using an informal proof as a guide
- Iteratively refine the proof based on Lean feedback
 - Can be seen as self-correction through in-context learning
 - Akin to how humans improve their understanding based on feedback

3 Final Proof Synthesis Guided by Verified Lemmas and Iterative Feedback

- Consider overall proof using the lemmas
 - Use only the verified lemmas
- Allows bottom-up strategy construction even when the full plan isn't initially clear
 - Prior work: top-down approach requiring the full plan upfront
- Iteratively refine the proof based on Lean feedback

Experiments

Experimental Setup

- Informal LLM: DeepSeek-R1-0528-Qwen3-8B
- Formal prover model: DeepSeek-Prover-V2-7B
- AutoFormalizer: Kimina-Autoformalizer-7B

Comparison of Formal Theorem-Proving Performance

Prover System	Method	Model Size	Sample Budget	miniF2F test
Large Language Models				
Kimina-Prover-Preview (Wang et al., 2025)	Whole-proof	72B	1	52.9%
			32	68.9%
			1024	77.9%
			8192	80.7%
DeepSeek-Prover-V2 (non-CoT) (Ren et al., 2025)	Whole-proof	671B	1	59.5%
			32	73.8%
			1024	76.7%
			8192	78.3%
DeepSeek-Prover-V2 (CoT) (Ren et al., 2025)	Whole-proof	671B	1	61.9%
			32	82.4%
			1024	86.6%
			8192	88.9%
Small Language Models				
DeepSeek-Prover-V1.5-RL + RMaxTS (Xin et al., 2025a)	Tree search	7B	$32 \times 16 \times 400$	63.5%
InternLM2.5-StepProver + BFS + CG (Wu et al., 2024)	Tree search	7B	$256 \times 32 \times 600$	65.9%
HunyuanProver v16 + BFS + DC (Li et al., 2025)	Tree search	7B	$600 \times 8 \times 400$	68.4%
BFS-Prover (Xin et al., 2025b)	Tree search	7B	$2048 \times 2 \times 600$	70.8%
Leanabell-Prover-GD-RL (Zhang et al., 2025)	Whole-proof	7B	128	61.1%
Goedel-Prover-SFT (Lin et al., 2025)	Whole-proof	7B	25600	64.7%
STP (Dong & Ma, 2025)	Whole-proof	7B	25600	67.6%
Kimina-Prover-Preview-Distill (Wang et al., 2025)	Whole-proof	7B	1	52.5%
			32	63.1%
			1024	70.8%
			8192	75.0%
DeepSeek-Prover-V2 (non-CoT) (Ren et al., 2025)	Whole-proof	7B	1	55.5%
			32	68.0%
			1024	73.2%
			8192	75.0%
DeepSeek-Prover-V2 (CoT) (Ren et al., 2025)	Whole-proof	7B	1	58.6%
			32	75.6%
			1024	79.9%
			8192	82.0%
Prover Agent (Ours)	Agent	8B	1	61.5%
			100	80.7%
			400	84.0%
			2000	86.1%

- ✓ State-of-the-art performance among methods using SLMs
- ✓ High success rate under low sample budget
- ✓ Better performance than prior work through coordination

Performance on Olympiad-Level Problems

		Model Size	Sample Budget	Olympiad				MATH			Custom			
				IMO	AIME	AMC	Sum	Algebra	Number Theory	Sum	Algebra	Number Theory	Induction	Sum
Number of Problems				20	15	45	80	70	60	130	18	8	8	34
Prover Agent (Ours)	(Direct proving w/o iterative refinement)	8B	1	40.0	53.3	62.2	55.0	71.4	60.0	66.2	55.6	75.0	50.0	58.8
	(Direct proving w/o iterative refinement)		100	70.0	80.0	82.2	78.8	82.9	88.3	85.4	66.7	75.0	62.5	67.6
	(Direct proving w/ iterative refinement)		400	80.0	80.0	88.9	85.0	84.3	91.7	87.7	66.7	75.0	62.5	67.6
	(Final proof synthesis w/ lemma)		2000	80.0	80.0	91.1	86.3	85.7	91.7	88.5	72.2	87.5	75.0	76.5
DeepSeek-Prover-V2 (Ren et al., 2025)		671B	8192	50.0	93.3	77.8	73.8	100.0	96.7	98.5	83.3	87.5	100.0	88.2

- Show strong performance on Olympiad-level problems
 - ➡ Suggest that coordination with informal reasoning may be the key
 - Olympiad-level problems require a high degree of mathematical reasoning
- Consistent gap in MATH and Custom
 - ➡ Suggests that model size and sample budget may play a more significant role here
 - Prover model also possesses a certain level of mathematical reasoning ability

Case Study: Success with Lemma-Guided Proofs

Problem:

```
1 theorem induction_nfactltnexpnmlngt3
2   (n : ℕ)
3   (h0 : 3 ≤ n) :
4   (n)! < n^(n-1) := by sorry
```

Reasoning trace w/ lemmas:

- ✓ Consider the specific cases for $n = 3, 4, 5$
 - ➡ Clearly identify the use of mathematical induction
- ✓ Employ proof techniques used in the lemmas

Reasoning trace w/o lemmas:

- ✗ Proof strategy is unclear
- ✗ The details cannot be worked out sufficiently

Generated lemmas:

```
1 theorem base_case_3 : (3 : ℕ)! < 3^(3-1) := by
2   have h_main : (3 : ℕ)! < 3^(3-1) := by
3     -- Calculate the factorial and the power step-by-step
4     norm_num [Nat.factorial, Nat.pow_succ, Nat.mul_assoc]
5     -- Use 'decide' to confirm the inequality
6     <=> decide
7   exact h_main
```

```
1 theorem exponent_inequality {n : ℕ} (hn : 2 ≤ n) :
2   n^(n-1) < (n+1)^(n-1) := by
3   have h_main : n^(n-1) < (n+1)^(n-1) := by
4     have h0 : n-1+1 = n := by
5       omega
6     have h1 : n ≥ 2 := by omega
7     have h2 : n-1 ≥ 1 := by omega
8     have h3 : n < n+1 := by omega
9     -- Use the fact that if a < b and n > 0, then a^n < b^n
10    exact calc
11      n^(n-1) < (n+1)^(n-1) := by
12        -- Apply the lemma that if a < b and n > 0, then
13        a^n < b^n
14      exact Nat.pow_lt_pow_of_lt_left h3 (by omega)
15    _ = (n+1)^(n-1) := by rfl
16  exact h_main
```

The full reasoning trace can be found in the appendix of our paper.